

Task 9: NumPy Arrays and Vectorized Computing

Track: Data Science | Level 1 - Day 9 | Tools: Python, NumPy, Jupyter Notebook

Level 1 - Day 9: NumPy Arrays and Vectorized Computing

Track: Data Science

Task: Task 9 - Numerical Computing with NumPy Arrays

Objective: Master NumPy N-dimensional array (`ndarray`) creation, multi-dimensional indexing, slicing, broadcasting, element-wise math, axis-based statistical aggregations, and reshaping for data science preprocessing.

1. Quick Reference: Python Lists vs NumPy Arrays

Feature	Python Standard List	NumPy `ndarray`
Memory Layout	Array of pointers to fragmented heap objects	Contiguous memory block of homogenous primitives
Vectorized Computation	Requires explicit loops or comprehensions	Direct C-level SIMD vectorized operations
Dimensionality	Nested lists of varying lengths	Strict N-dimensional tensors with uniform shape
Performance	General-purpose, higher overhead	Fast numerical linear algebra and vectorized math

2. Core Array Operations & Methods (8 Practical Scenarios)

In [code]

```
import numpy as np

# Example 1: Array Initialization Methods (zeros, ones, arange, linspace)
zeros_arr = np.zeros(5)
ones_matrix = np.ones((2, 3), dtype=np.int32)
stepped_range = np.arange(10, 30, 5)
linear_spaced = np.linspace(0.0, 1.0, 5)

print(f"Ex 1a - Zeros Array: {zeros_arr}")
print(f"Ex 1b - Ones Matrix (2x3):\n{ones_matrix}")
print(f"Ex 1c - Stepped Range: {stepped_range}")
print(f"Ex 1d - Linspace Spaced Points: {linear_spaced}")
```

In [code]

```
# Example 2: Array Attributes (ndim, shape, size, dtype)
sample_array = np.array([[1.5, 2.8, 3.1], [4.9, 5.2, 6.7]], dtype=np.float32)

print("Ex 2 - Array Attribute Inspection:")
print(f" Dimensions (ndim): {sample_array.ndim}")
print(f" Shape: {sample_array.shape}")
print(f" Total Elements (size): {sample_array.size}")
print(f" Data Type (dtype): {sample_array.dtype}")
print(f" Bytes per Item (itemsize): {sample_array.itemsize} bytes")
```

In [code]

```
# Example 3: 2D Multi-dimensional Slicing and Striding
data_grid = np.array([
    [10, 11, 12, 13],
    [20, 21, 22, 23],
    [30, 31, 32, 33],
    [40, 41, 42, 43]
])

center_subgrid = data_grid[1:3, 1:3]
last_column = data_grid[:, -1]

print(f"Ex 3a - Original Grid (4x4):\n{data_grid}")
print(f"Ex 3b - Center 2x2 Subgrid:\n{center_subgrid}")
print(f"Ex 3c - Extracted Last Column: {last_column}")
```

In [code]

```
# Example 4: Boolean Masking & Conditional Filtering
# Isolate outliers in an environmental noise dataset (threshold > 75 dB)
noise_readings = np.array([45, 62, 78, 55, 89, 71, 95, 60, 83])
outlier_mask = noise_readings > 75
extreme_readings = noise_readings[outlier_mask]

print(f"Ex 4a - Boolean Filter Mask: {outlier_mask}")
print(f"Ex 4b - Extreme Noise Outliers (> 75 dB): {extreme_readings}")
```

In [code]

```
# Example 5: Vectorized Arithmetic & Broadcasting
# Adding a 1D normalization bias vector to each row of a matrix
feature_matrix = np.array([
    [100.0, 20.0],
    [150.0, 25.0],
    [200.0, 30.0]
])
bias_vector = np.array([5.0, 2.5])

adjusted_features = feature_matrix + bias_vector
scaled_features = feature_matrix * 1.10 # Scalar broadcasting

print(f"Ex 5a - Bias Adjusted Features (Vector Broadcasting):\n{adjusted_features}")
print(f"Ex 5b - Scaled Features (+10% Scalar Broadcasting):\n{scaled_features}")
```

In [code]

```
# Example 6: Statistical & Axis-based Aggregations (mean, std, sum, min, max)
exam_scores = np.array([
    [85, 90, 78],
    [60, 58, 64],
    [92, 95, 96],
    [45, 52, 48]
])

overall_mean = np.mean(exam_scores)
subject_means = np.mean(exam_scores, axis=0) # across columns
student_averages = np.mean(exam_scores, axis=1) # across rows
subject_std = np.std(exam_scores, axis=0)

print(f"Ex 6a - Overall Cohort Mean: {overall_mean:.2f}")
print(f"Ex 6b - Subject-wise Means (axis=0): {subject_means}")
print(f"Ex 6c - Student-wise Means (axis=1): {student_averages}")
print(f"Ex 6d - Subject Standard Deviations: {np.round(subject_std, 2)}")
```

In [code]

```
# Example 7: Array Reshaping, Flattening, and Transposition
flat_sensor_stream = np.arange(1, 13)
grid_3x4 = flat_sensor_stream.reshape((3, 4))
transposed_grid = grid_3x4.T
flattened_back = grid_3x4.ravel()

print(f"Ex 7a - Original Vector (12,): {flat_sensor_stream}")
print(f"Ex 7b - Reshaped to (3, 4):\n{grid_3x4}")
print(f"Ex 7c - Transposed Grid (4, 3):\n{transposed_grid}")
print(f"Ex 7d - Flattened Back (ravel): {flattened_back}")
```

In [code]

```
# Example 8: Conditional Array Transformation with np.where()
# Convert test marks into binary Pass/Fail flags (passing threshold = 60)
scores = np.array([75, 42, 88, 59, 94, 33, 61])
pass_fail_labels = np.where(scores >= 60, "Pass", "Fail")
curved_scores = np.where(scores < 50, scores + 5, scores)

print(f"Ex 8a - Raw Scores: {scores}")
print(f"Ex 8b - Binary Labels via np.where: {pass_fail_labels}")
print(f"Ex 8c - Grace Mark Curved Scores (<50 get +5): {curved_scores}")
```

3. End-to-End Vectorized Preprocessing Pipelines

Applied numerical array workflows demonstrating feature scaling (Min-Max Normalization & Z-score Standardization) and multi-feature analytics.

In [code]

```
# Pipeline 1: Vectorized Feature Normalization & Standardization Pipeline
# Raw student feature matrix: [Study Hours, Attendance Pct, Practice Exams Taken]
student_features = np.array([
    [12.5, 90.0, 4.0],
    [4.0, 65.0, 1.0],
    [18.0, 95.0, 6.0],
    [8.0, 75.0, 2.0],
    [15.5, 88.0, 5.0]
], dtype=np.float64)

# 1. Min-Max Normalization: (X - min) / (max - min)
feat_min = np.min(student_features, axis=0)
feat_max = np.max(student_features, axis=0)
min_max_scaled = (student_features - feat_min) / (feat_max - feat_min)

# 2. Z-Score Standardization: (X - mean) / std
feat_mean = np.mean(student_features, axis=0)
feat_std = np.std(student_features, axis=0)
z_score_standardized = (student_features - feat_mean) / feat_std

print("=== STUDENT FEATURE SCALING PIPELINE REPORT ===")
print(f"Feature Minimums (axis=0): {feat_min}")
print(f"Feature Maximums (axis=0): {feat_max}")
print(f"\nMin-Max Normalized Features (Scale: [0, 1]):\n{np.round(min_max_scaled, 3)}")
print(f"\nZ-Score Standardized Features (Mean=0, Std=1):\n{np.round(z_score_standardized, 3)}")
```

In [code]

```
# Pipeline 2: Employee Payroll & Performance Matrix Analytics
# Columns: [Employee ID, Base Salary, Rating (1-5), Years Experience]
employee_data = np.array([
    [101, 115000, 4.8, 6.0],
    [102, 45000, 3.2, 1.5],
    [103, 82000, 4.2, 4.0],
    [104, 52000, 4.6, 5.0],
    [105, 48000, 3.9, 2.0]
])

emp_ids = employee_data[:, 0].astype(int)
salaries = employee_data[:, 1]
ratings = employee_data[:, 2]
experience = employee_data[:, 3]

# Compute Vectorized Appraisal Hike: 12% if rating >= 4.5, else 6%
hike_percentages = np.where(ratings >= 4.5, 0.12, 0.06)
revised_salaries = salaries * (1.0 + hike_percentages)

# Matrix Analytics Summary
print("\n=== EMPLOYEE COMPENSATION & VECTORIZED APPRAISAL ===")
for i in range(len(emp_ids)):
    print(f"Emp ID: {emp_ids[i]} | Rating: {ratings[i]:.1f} | Base: ${salaries[i]:,.0f} -> Revised: $
{revised_salaries[i]:,.2f} ({hike_percentages[i]*100:.0f}%)")

total_payroll_initial = np.sum(salaries)
total_payroll_revised = np.sum(revised_salaries)
payroll_delta = total_payroll_revised - total_payroll_initial

print(f"\nTotal Initial Payroll: ${total_payroll_initial:,.2f}")
print(f"Total Revised Payroll: ${total_payroll_revised:,.2f}")
print(f"Disbursement Increment: ${payroll_delta:,.2f} ({(payroll_delta / total_payroll_initial) *
100:.2f}%)")
```